

Max Shuford

CS470.001

Prof. Zhu

2/3/2026

Process Management in Unix Operating Systems

1. Introduction

Process management is an important part of an operating system because it controls how programs run and how system resources are shared. A process is a program that is currently being executed, and the operating system needs to manage many processes at the same time. Process management makes it possible for multiple programs to run concurrently while keeping the system stable and responsive.

2. Implementation Summary

The program is structured around a parent process that creates multiple child processes using a loop with the `fork()` system call. At the beginning of the program, the parent prints its own process ID (PID). Each time `fork()` is called, a new child process is created, and the parent stores the child's PID in an array in the order the children are created. Each child process prints its index, its PID, and the command it is about to run. The child then uses `execvp()` to execute a Linux command. Some child processes run valid commands, while others intentionally attempt to run invalid commands to show how execution failures work. Two of the child processes intentionally terminate using `abort()` to demonstrate signal-based termination. Once all child processes have been created, the parent process waits for them to finish using `waitpid()`.

3. Results and Observations

The child processes are created one at a time in a loop, and each child receives a unique PID from the operating system. These PIDs are saved by the parent in an array so that each child process can be tracked later.

There is an important difference between process creation order and process termination order. Process creation order is the order in which the parent creates the child processes using `fork()`. Process termination order depends on how long each child process takes to execute, so it can be different from the creation order. In this program, the parent explicitly waits for the child processes in creation order by calling `waitpid()` on each stored PID.

The parent determines how each child process ended by examining the status value returned by `waitpid()`. If the child exited normally, the parent checks the exit code. If the child was terminated by a signal, the parent identifies which signal caused the termination. This makes it possible to distinguish between successful execution, execution errors, and abnormal termination caused by signals.

4. Conclusion

This lab helped reinforce how process management works in Unix-based operating systems. By using `fork()`, `execvp()`, and `waitpid()`, the program demonstrated how processes are created, executed, and synchronized. It also showed the difference between normal process termination and signal-based termination. Overall, this assignment provided a better understanding of how the operating system manages multiple processes and tracks their execution status.